

Design Patterns Used in the Park Management System

1. MVC Pattern (Model-View-Controller)

Name: Model-View-Controller (MVC)

Intent: The MVC pattern is used to separate the application into three main parts: the user interface, the business logic, and the data layer. This makes the system easier to organize, maintain, and update in the future.

Problem: In the park management system, some features such as volunteer registration and service-hour tracking contain a lot of logic and validation rules. If all this logic is written directly inside the interface page, the code becomes difficult to maintain and any future modification may affect the whole page.

Solution: The system uses a Controller as a middle layer between the View and the Model. The volunteer.php page sends the request to the VolunteerController, and the controller handles all validations and business logic before returning the result to the interface.

Participants: - VolunteerController: Responsible for processing volunteer actions such as joining or cancelling shifts.

- Volunteer View (volunteer.php): Displays available shifts, volunteer information, and service-hour statistics.

Implementation: // Inspired by volunteer.php
\$controller = new VolunteerController();

// Request volunteer statistics
\$userStats = \$controller->serviceHourTracker(\$userId);

// Join a volunteer shift
\$result = \$controller->joinShift(\$userId, \$shiftId, \$userRole);

Benefits: Separates interface code from application logic, improves code readability, and makes the system easier to maintain.

2. Observer Pattern (Weather & Status Notifications)

Name: Observer Pattern

Intent: The Observer pattern is used to notify related users or components automatically whenever an important event happens in the system.

Problem: Volunteers can report dangerous weather conditions such as heavy rain or extreme heat. Administrators must receive these reports immediately in order to take proper action.

Solution: The weather reporting feature works as the Subject, while the Admin system acts as the Observer. Whenever a volunteer submits a weather report, the administration is automatically notified.

Participants: - Subject: Weather reports submitted from volunteer.php.
- Observer: Admin dashboard or notification system that receives weather alerts.

Implementation: // From volunteer.php
if (isset(\$_POST['action']) && \$_POST['action'] === 'weather_override') {

 // Send weather report to the administrator
 \$controller->reportWeather(\$userId, \$weather, \$date);
}

Benefits: Provides faster communication between volunteers and administrators and improves safety management.

3. State Pattern (Shift Lifecycle)

Name: State Pattern

Intent: The State pattern allows the system to change its behavior depending on the current state of the shift.

Problem: A volunteer cannot join a closed shift and cannot cancel a shift that was never joined. Managing these rules manually everywhere makes the code harder to maintain.

Solution: The system checks the current shift state before displaying actions to the user. Depending on the state stored in the database, the interface decides whether the volunteer can join the shift or not.

Participants: - Shift Status: Determines whether the shift is open or closed.
- Join State: Checks if the volunteer already joined the shift.

Implementation: <?php if (\$shift['status'] == 'open'): ?>

<?php if (\$joined): ?>

<button disabled>Joined</button>

<?php else: ?>

<button class="btn-success">Join</button>

<?php endif; ?>

<?php else: ?>

<button class="btn-dark" disabled>Closed</button>

```
<?php endif; ?>
```

Benefits: Makes the interface more dynamic, prevents invalid actions, and simplifies shift management logic.